

Dari Data Menuju Model Prediktif

Course:

Matrikulasi S3 Ilmu Komputer FILKOM UB

by:

Yuita Arum Sari, S.Kom., M.Kom., Ph.D.

yuita@ub.ac.id

Versi Dokumen:

Tanggal: 15; Bulan: September; Tahun: 2026

© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id

Learning outcomes

Setelah sesi ini, mahasiswa seharusnya mampu menjelaskan “mengapa” di balik training model, bukan hanya menjalankan *library*.



1 - Formulate

Mengubah kasus prediksi menjadi $Xw \approx y$ dan memilih bentuk model yang sesuai.

- Menulis model dalam notasi matriks
- Menafsirkan residual



2 - Optimize

Menurunkan loss, memahami gradien, learning rate, dan dinamika gradient descent.

MAMPU

- Menghitung satu langkah GD secara manual
- Mendiagnosis learning rate & scaling

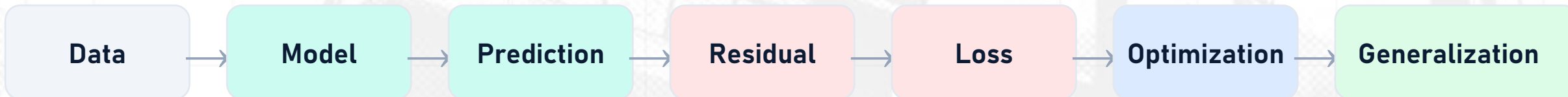


3 - Generalize

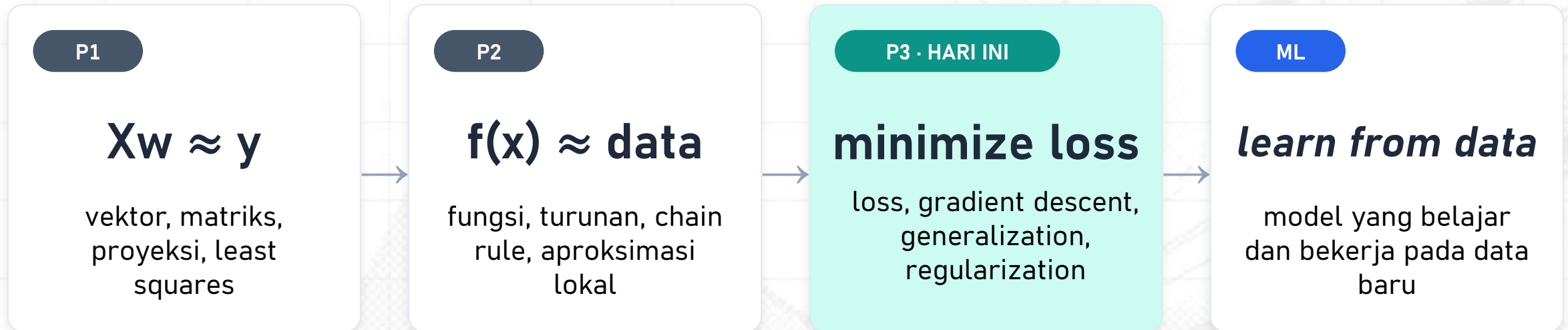
Membedakan fit vs generalization; menjelaskan regularisasi sebagai kontrol kompleksitas.

MAMPU

- Merancang split yang jujur
- Memilih λ dengan validasi



Bagaimana Membuat Prediksi yang Baik?



Bagaimana komputer memilih parameter model yang membuat prediksi baik — dan tetap baik pada data yang belum pernah dilihat?

Notasi dan konvensi

Simbol berikut dipakai konsisten sepanjang materi..

 n

jumlah sampel
skalar

 p

jumlah fitur
skalar

 $x_i \in \mathbb{R}^p$

input sampel ke-i
vektor fitur

 $X \in \mathbb{R}^{n \times p}$

matriks desain
n baris, p kolom

 $y \in \mathbb{R}^n$

target / label
vektor

 w, θ

parameter model
dipelajari dari data

 $\hat{y} = f(x; \theta)$

prediksi model
output

 $r = y - \hat{y}$

residual
sisanya yang belum dijelaskan

 $L(\theta)$

loss function
skalar ≥ 0

 ∇L

gradien loss
vektor, arah naik tercepat

 η

learning rate
hyperparameter

 λ

kekuatan regularisasi
hyperparameter

Kasus utama: jam belajar → nilai ujian

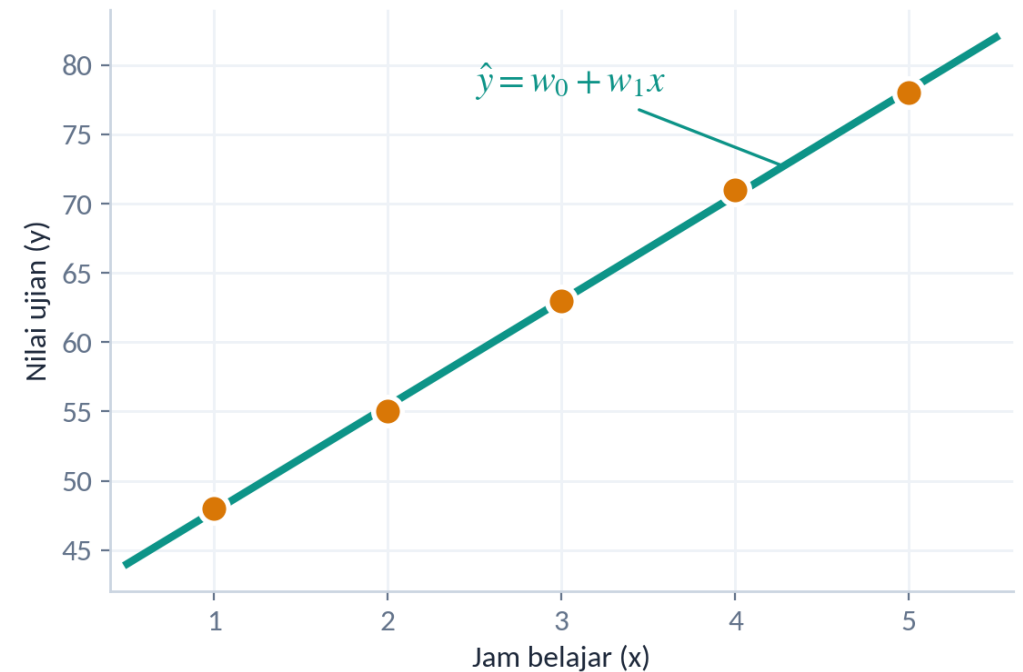
Sengaja sederhana agar seluruh mekanisme training terlihat tanpa tertutup kompleksitas domain.

x: jam	1	2	3	4	5
y: nilai	48	55	63	71	78

$$\hat{y} = w_0 + w_1x$$

Tujuan: cari w_0 dan w_1 sehingga garis prediksi sedekat mungkin dengan data.

Regresi contoh paling transparan dari proses belajar: memilih parameter berdasarkan suatu kriteria kesalahan.



01

MODEL

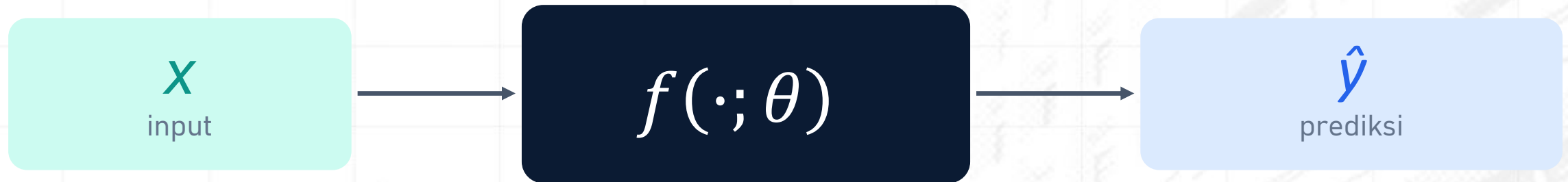
Regression as a Model

$$\hat{y} = f(x; \theta)$$

- 1 Model = fungsi berparameter
- 2 Linear dalam parameter
- 3 Dari satu fitur ke p fitur; bentuk matriks
- 4 Residual: sisa yang belum dijelaskan

Apa sebenarnya sebuah model?

Model adalah fungsi berparameter yang memetakan input menjadi prediksi.



θ : parameter dipelajari dari data



x input

Fitur yang diketahui:
jam belajar, usia, pixel,
embedding, sensor,
dst.



θ parameter

Nilai yang dipelajari
dari data: bobot dan
bias.



$\hat{y}$ — prediction

Output model; belum
tentu sama dengan
target y.



Training

Proses memilih θ agar
prediksi memenuhi
objective.

Machine learning = memilih θ dari data, bukan menulis θ secara manual.

Linear regression: “linear” terhadap parameter

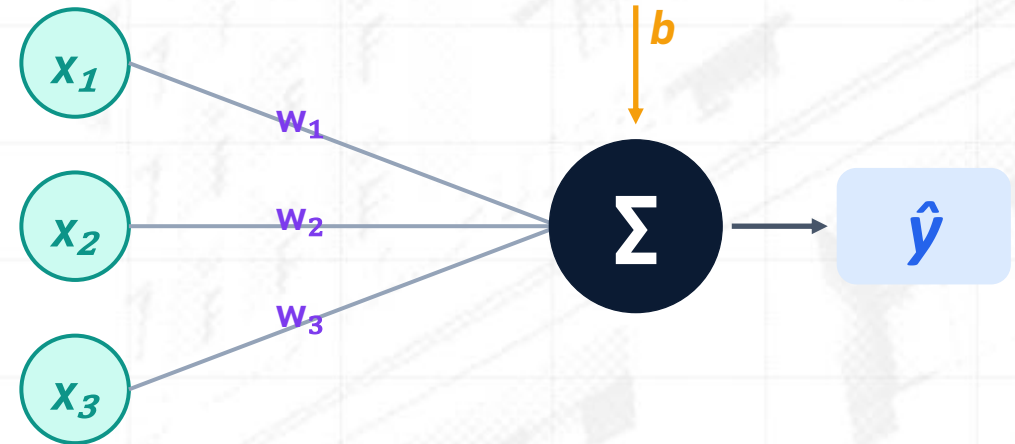
Bentuk paling sederhana, tetapi struktur matematisnya muncul lagi pada model jauh lebih kompleks.

$$\hat{y}_i = w_0 + w_1 x_i$$

$$\hat{y} = Xw$$

Dengan bias dimasukkan sebagai kolom 1:

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \quad W = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$



$$z = w^T x + b$$



Linear terhadap w, bukan terhadap x

$\hat{y} = w_0 + w_1 x + w_2 x^2$ tetap linear regression: ia linear dalam w.



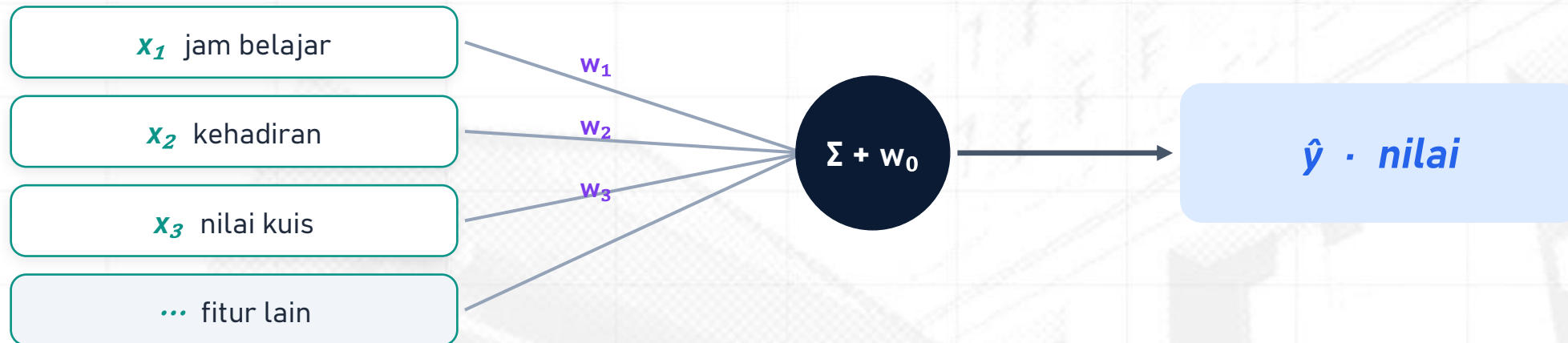
Koneksi ke neural network

Satu neuron linear juga menghitung $z = w^T x + b$. Regresi linear adalah “unit komputasi” dasar tanpa aktivasi nonlinear.

Dari satu fitur ke banyak fitur

Pada data nyata, satu target biasanya dijelaskan oleh banyak variabel sekaligus.

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_px_p$$



Interpretasi koefisien

w_j mengukur perubahan prediksi akibat perubahan x_j sebesar satu unit ketika fitur lain dianggap tetap. Interpretasi ini bergantung pada skala, korelasi fitur, dan asumsi model.

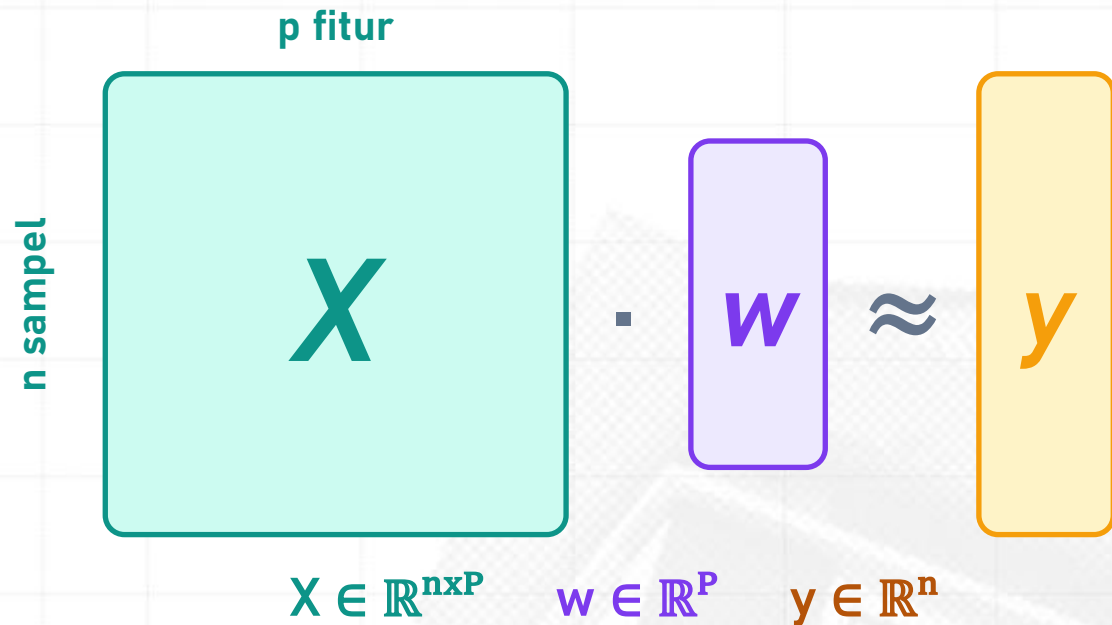


Hati-hati: multikolinearitas

Fitur yang sangat berkorelasi membuat koefisien tidak stabil. Ini salah satu motivasi ridge (bagian 7).

Matrix view: seluruh dataset dalam satu persamaan

Notasi matriks membuat hubungan dengan aljabar linear dan implementasi komputasional menjadi eksplisit.



$$y \approx Xw$$

Mengapa tidak ada solusi eksak?

- $n \gg p$: sistem overdetermined, persamaan lebih banyak daripada variabel
- Noise pengukuran pada y
- Bentuk model tidak sempurna

$$r = y - Xw$$



Karena itu masalah berubah dari "menyelesaikan persamaan" menjadi **"mencari pendekatan terbaik"**.

Prediksi $\neq$ target: residual membawa informasi

Residual menunjukkan bagian data yang belum dijelaskan oleh model.

x	y	$\hat{y}$	$r = y - \hat{y}$
1	48	49	-1
2	55	56	-1
3	63	63	0
4	71	70	1
5	78	77	1

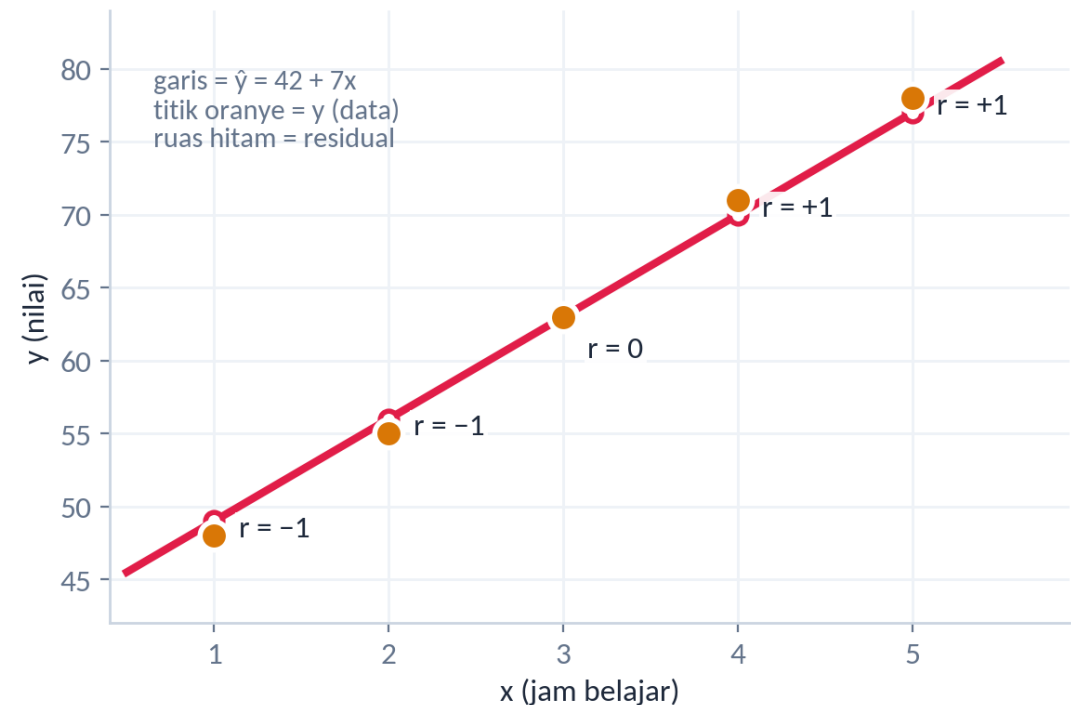
$$r_i = y_i - \hat{y}_i$$

Residual pattern

Jika residual memiliki pola terhadap x , model mungkin kehilangan struktur penting.

Residual magnitude

Semakin besar $|r_i|$, semakin jauh prediksi dari observasi sampel i .

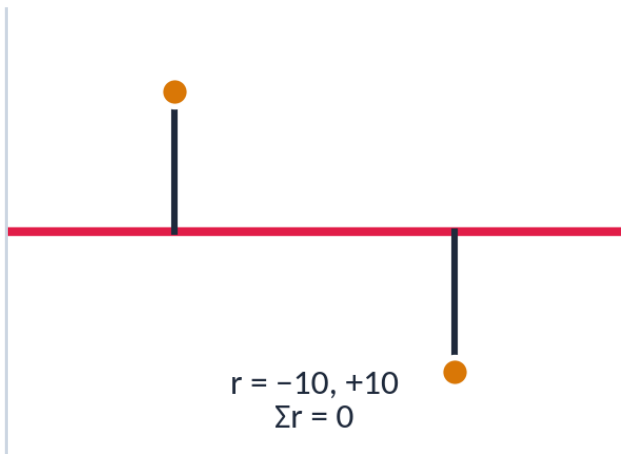


Mengapa tidak meminimalkan jumlah residual saja?

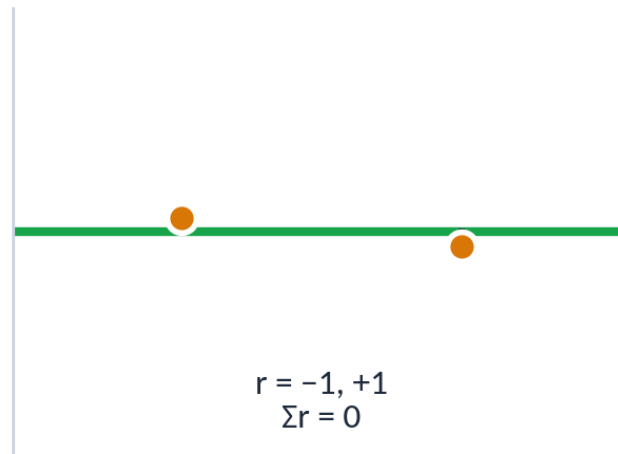
Residual positif dan negatif dapat saling meniadakan.

$\sum r_i = 0$ tidak berarti prediksi sempurna

Model A



Model B



Model A

Residual: $-10, +10 \rightarrow \sum r = 0$, tetapi error besar.



Model B

Residual: $-1, +1 \rightarrow \sum r = 0$, dan jauh lebih dekat.

Kita butuh fungsi yang mengubah kumpulan residual menjadi satu angka yang mencerminkan “buruknya” model.

02

LOSS

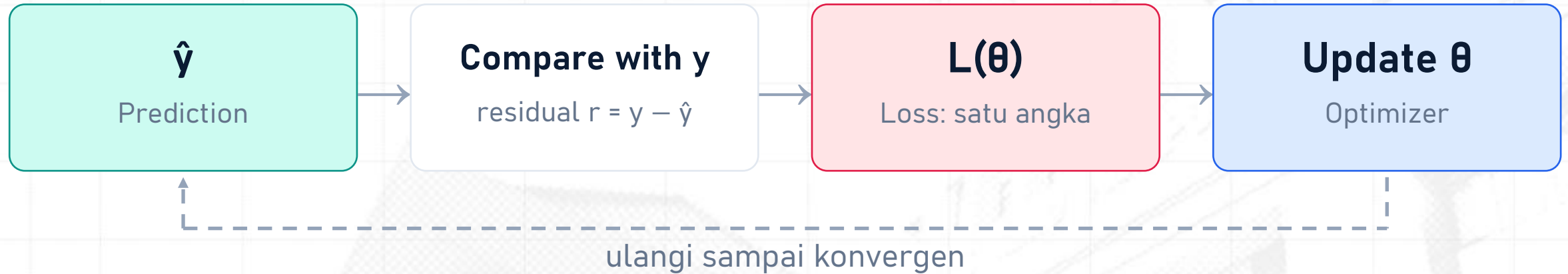
Loss: Turning Error into an Objective

$L(\theta)$ = measure of prediction error

- 1 Squared error dan MSE
- 2 MSE vs MAE vs Huber
- 3 Loss sebagai log-likelihood
- 4 Normal equation & geometri
- 5 Loss $\neq$ metric $\neq$ tujuan riset

Loss function adalah definisi matematis dari “baik”

Training hanya dapat mengoptimasi apa yang kita definisikan sebagai objective.



Objective menentukan perilaku model

Mengubah loss dapat mengubah solusi meskipun data dan arsitektur sama.

1 MSE

Prediksi mendekati rata-rata bersyarat $E[y|x]$

2 MAE

Prediksi mendekati median bersyarat, lebih tahan outlier

3 Cross-entropy

Prediksi mendekati probabilitas kelas

Squared error: mengapa dikuadratkan?

Kuadrat membuat error non-negatif, memberi penalti lebih besar pada error besar, dan mudah didiferensialkan.

$$SE_i = (y_i - \hat{y}_i)^2$$

$$MSE = (1/n) \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Non-negative

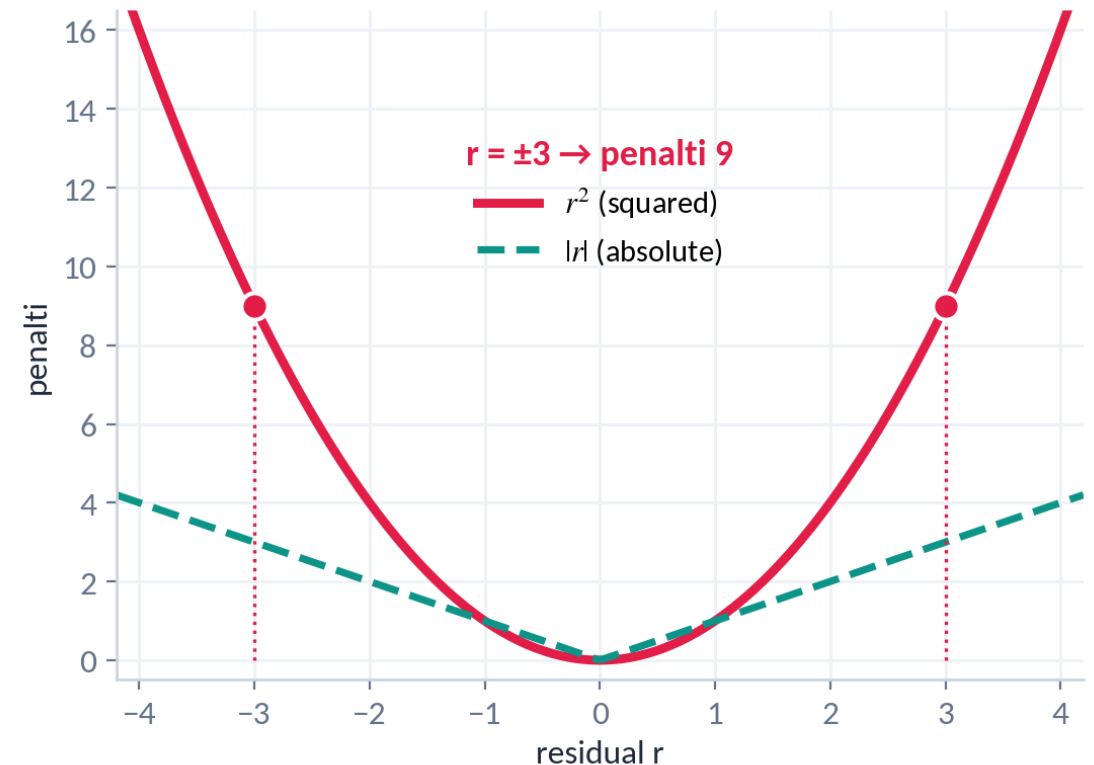
Error +3 dan -3 sama-sama memberi penalti 9.

Smooth

Turunan kontinu, nyaman untuk gradient-based optimization.

Outlier-sensitive

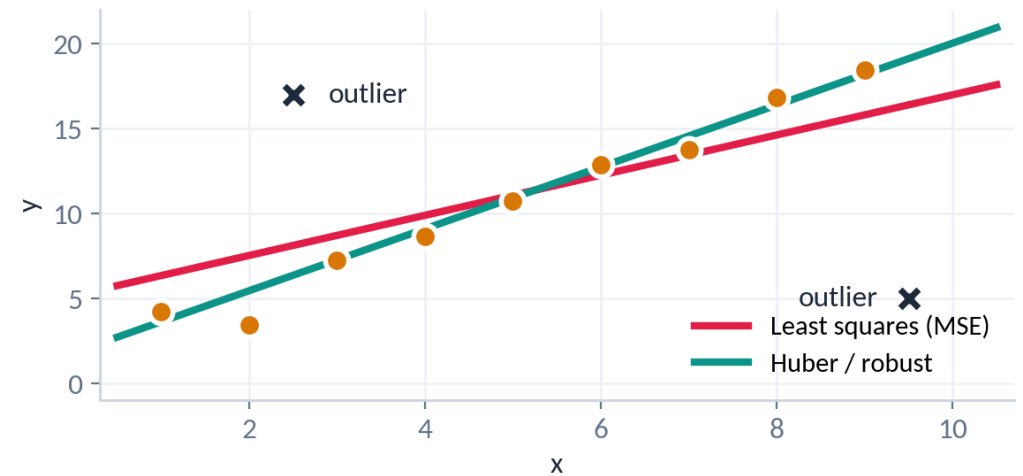
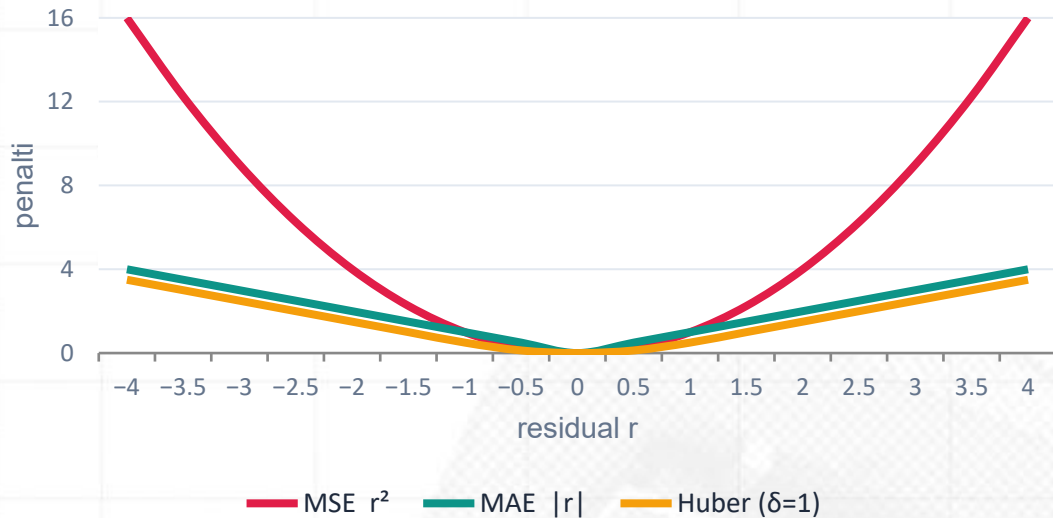
Residual 10 memberi loss 100; residual 2 hanya 4.



MSE bukan satu-satunya pilihan; ia membawa asumsi dan konsekuensi.

MSE vs MAE vs Huber

Bentuk kurva penalti menentukan seberapa besar pengaruh outlier terhadap solusi.



Loss	Rumus per sampel	Optimum populasi	Outlier	Turunan
MSE	r^2	mean bersyarat	sangat sensitif	mulus
MAE	$ r $	median bersyarat	tahan	patah di $r = 0$
Huber	$\frac{1}{2}r^2$ jika $ r \leq \delta$, selain itu $\delta(r - \frac{1}{2}\delta)$	antara mean dan median	tahan	mulus

Loss sebagai *log-likelihood*: mengapa MSE?

Meminimalkan MSE setara dengan maximum likelihood bila noise diasumsikan Gaussian.

1 Model probabilistik

$$y = f(x; \theta) + \varepsilon, \varepsilon \sim N(0, \sigma^2)$$

↓

2 Likelihood satu sampel

$$p(y_i | x_i, \theta) = N(y_i; \hat{y}_i, \sigma^2)$$

↓

3 Negative log-likelihood

$$-\log p = (1/2\sigma^2) \sum (y_i - \hat{y}_i)^2 + \text{konstanta}$$

↓

4 Maximum likelihood

argmax likelihood $\Leftrightarrow$ argmin MSE

ASUMSI NOISE $\rightarrow$ LOSS

Distribusi noise	Loss yang muncul
Gaussian	MSE (squared error)
Laplace	MAE (absolute error)
Bernoulli	Cross-entropy (log loss)
Student-t / heavy tail	Robust loss (mis. Huber)

Implikasi riset

Memilih loss berarti memilih asumsi tentang noise. Jika data memiliki outlier berat, asumsi Gaussian tidak masuk akal.

MSE dalam bentuk matriks

Bentuk ini langsung menghubungkan regression dengan least squares dari P1.

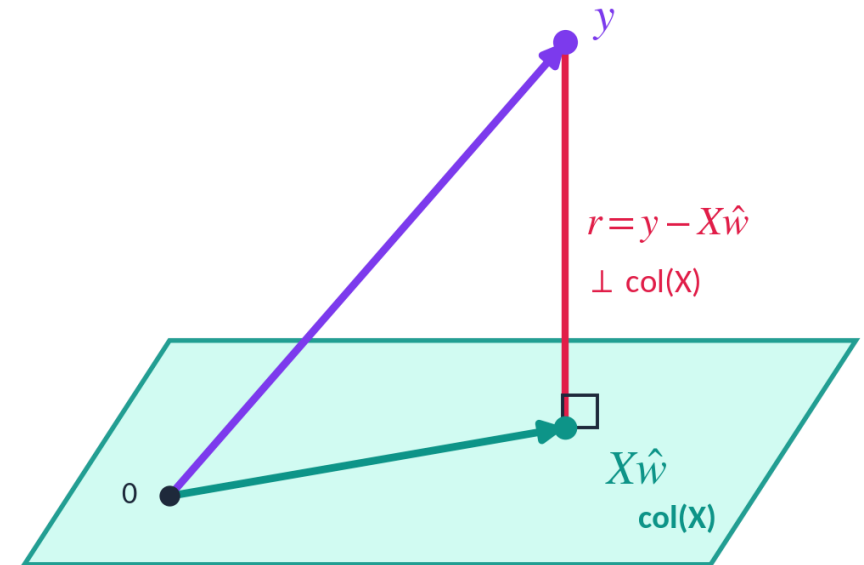
$$\begin{aligned} L(w) &= (1/n) \|y - Xw\|_2^2 \\ &= (1/n) (y - Xw)^T (y - Xw) \end{aligned}$$

Geometri

Kita mencari titik Xw pada column space X yang paling dekat dengan y dalam jarak Euclidean.

Koneksi P1

Normal equation dan gradient descent mengoptimasi objective yang sama; yang berbeda adalah cara memperoleh parameter.



$$r \perp \text{col}(X) \Rightarrow X^T r = 0$$

MSE → Least Squares: sebenarnya masalah yang sama

MSE hanya menuliskan jumlah kuadrat error secara ringkas dalam bentuk matriks.

1. Mulai dari MSE biasa

Nilai asli: $y = [5, 7, 10]$

Prediksi: $\hat{y} = [4, 8, 8]$

Error: $y - \hat{y} = [1, -1, 2]$

$$\text{MSE} = \frac{1}{3}[(1)^2 + (-1)^2 + (2)^2]$$
$$= 2$$

Intinya: kuadratkan semua error, lalu ambil rata-ratanya.

2. Regresi linear

Prediksi model:

$$\hat{y} = Xw$$

maka error = $y - Xw$

$$L(w) = \frac{1}{n} \|y - Xw\|_2^2$$

$$\|e\|^2 = e^T e$$

Jadi ini tetap “jumlah kuadrat error” — hanya ditulis sekaligus sebagai vektor.

3. Cari w terbaik

minimize

$$\min_w \|y - Xw\|_2^2$$

w

= Least Squares

Solusi analitiknya:

$$X^T X w = X^T y$$

Pesan utama: MSE pada regresi linear = least squares. Yang berubah hanya cara penulisannya.

Derivasi *normal equation* dari *loss*

Solusi closed-form muncul ketika gradien loss terhadap w dibuat nol.

Objective

1

$$L(w) = \|y - Xw\|^2$$

Gradien

2

$$\nabla L(w) = -2X^T(y - Xw)$$

Set $\nabla L = 0$

3

$$X^T X w = X^T y$$

Solusi

4

$$\hat{w} = (X^T X)^{-1} X^T y$$



Catatan penting

Inverse eksplisit bukan pilihan numerik terbaik. Gunakan solver, QR, SVD, atau pseudo-inverse. Rumus ini terutama penting untuk memahami struktur solusi.



Kapan $X^T X$ invertible?

Bila kolom X bebas linear (full column rank).

Normal equation pada kasus jam belajar

Solusi eksak untuk data kita: jauh lebih baik daripada tebakan awal $42 + 7x$.

$$X^T X = \begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix} \quad X^T y = \begin{bmatrix} 315 \\ 1021 \end{bmatrix}$$

$$\hat{w} = (X^T X)^{-1} X^T y$$

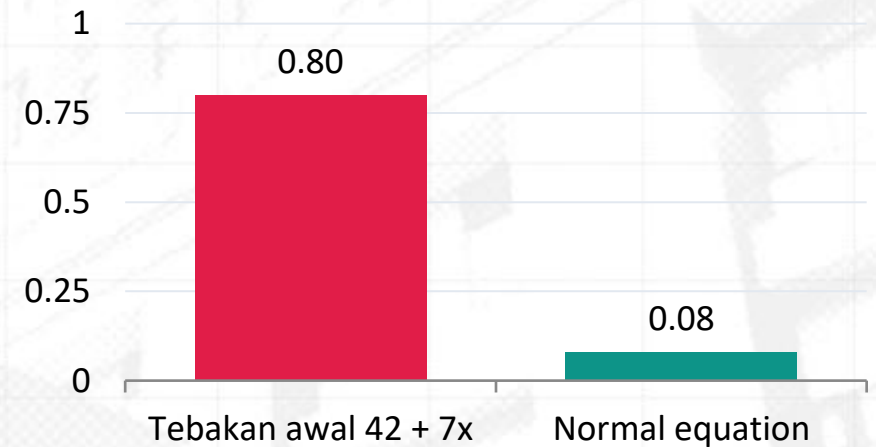
$$w_0 = (55 \cdot 315 - 15 \cdot 1021) / 50 = 40.2$$

$$w_1 = (-15 \cdot 315 + 5 \cdot 1021) / 50 = 7.6$$

x	y	$\hat{y}$	r
1	48	47.8	+0.2
2	55	55.4	-0.4
3	63	63	0
4	71	70.6	+0.4
5	78	78.2	-0.2

$$\hat{y} = 40.2 + 7.6x$$

MSE pada data training



Setiap tambahan 1 jam belajar dikaitkan dengan kenaikan ≈ 7.6 poin pada data ini. MSE turun 10x menjadi 0.08.

Closed-form vs iterative optimization

Mengapa gradient descent diperlukan jika regresi linear punya solusi analitik?



Closed-form/direct

- Sangat berguna untuk problem kecil-menengah
- Memberi insight matematis yang jelas
- Dapat mahal ketika p besar; $X^T X$ dapat ill-conditioned
- Tidak tersedia untuk banyak model nonlinear

$$O(np^2 + p^3)$$

biaya normal equation (bentuk $X^T X$ lalu dekomposisi)



Iterative/gradient-based

- Update parameter sedikit demi sedikit
- Skalabel ke jutaan parameter; cocok untuk mini-batch / streaming
- Menjadi dasar training neural network
- Membutuhkan learning rate dan stopping criterion

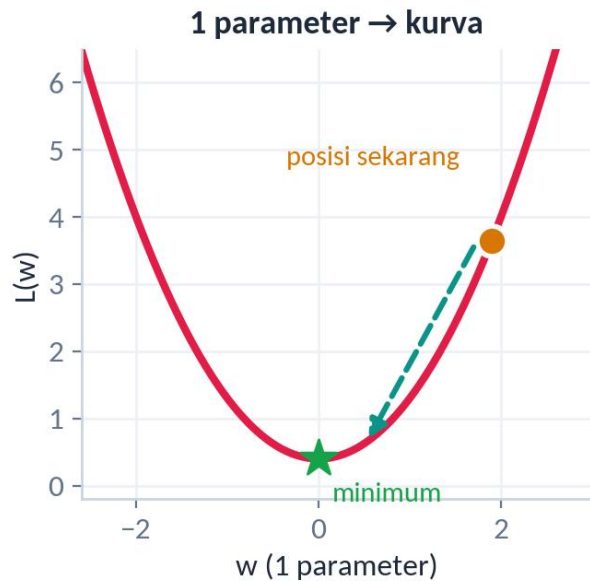
$$O(np)/iterasi$$

biaya satu langkah gradient descent penuh

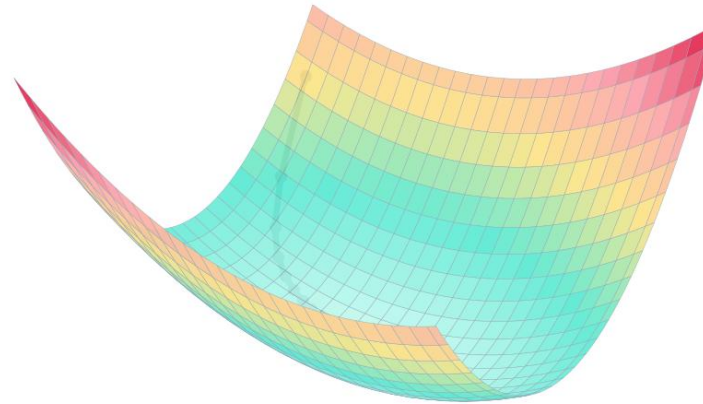
Gradient descent bukan “rumus lain untuk regresi”; ia adalah paradigma optimasi yang jauh lebih umum.

Loss landscape: parameter menjadi koordinat

Untuk satu parameter *loss* berupa kurva; untuk banyak parameter menjadi permukaan berdimensi tinggi.



banyak parameter → permukaan



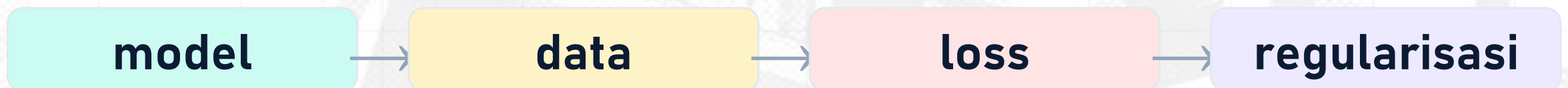
Training

Bergerak di ruang parameter untuk mencari nilai yang menurunkan *loss*.

Landscape

Bentuk permukaan menentukan mudah atau sulitnya optimasi.

Bentuk Landscape Ditentukan Oleh



Kasus numerik: hitung *loss* satu model

Gunakan model awal $\hat{y} = 42 + 7x$ pada data jam belajar.

x	y	$\hat{y} = 42 + 7x$	r	r^2
1	48	49	-1	1
2	55	56	-1	1
3	63	63	0	0
4	71	70	1	1
5	78	77	1	1
Σ				4

$$MSE = (1 + 1 + 0 + 1 + 1)/5 = 0.8$$

0.8

MSE model awal

≈ 0.89

RMSE = $\sqrt{0.8}$,
satuan poin nilai

Interpretasi

Loss = 0.8 adalah ringkasan error model pada data training. Nilainya bermakna relatif terhadap skala target dan model pembanding (mis. 0.08 untuk solusi optimal).

Yuita Arum Sari

Loss $\neq$ metric $\neq$ objective bisnis

Ketiganya sering dicampur, padahal fungsinya berbeda.

Training loss

Diferensiabel dan dipakai optimizer.

CONTOH

MSE · cross-entropy

Evaluation metric

Dipakai menilai performa model.

CONTOH

MAE · RMSE · R^2 · F1 · AUROC

Real-world utility

Biaya salah prediksi, fairness, latency, safety, interpretability.

CONTOH

biaya · risiko · dampak



Model dapat memiliki *loss* lebih kecil tetapi belum tentu lebih berguna secara ilmiah atau operasional. Laporkan metric yang relevan dengan tujuan, bukan hanya yang dioptimasi.

03

GRADIENT

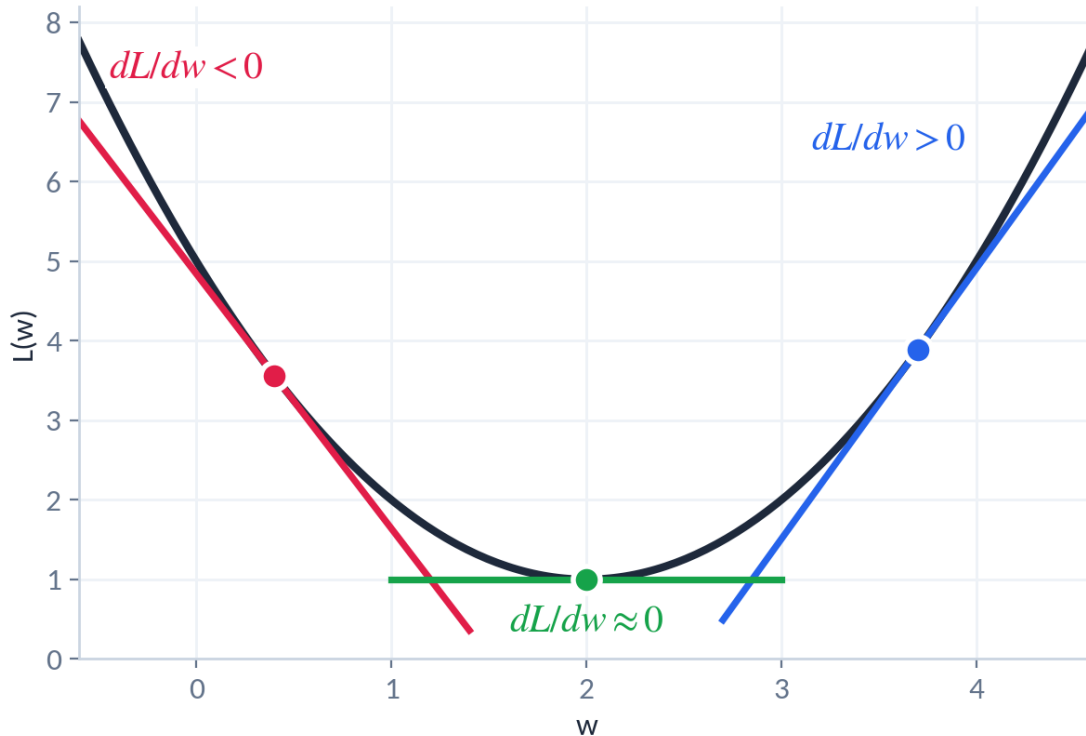
Gradient: Which Direction Changes the Loss?

$$\nabla L(\theta) = [\partial L / \partial \theta_1, \dots, \partial L / \partial \theta_p]T$$

- 1 Turunan sebagai sensitivitas
- 2 Gradien MSE regresi linear
- 3 Chain rule pada satu parameter
- 4 Chain rule → backpropagation
- 5 Gradien sebagai arah

Turunan sebagai sensitivitas

Derivative menjawab: jika parameter diubah sedikit, bagaimana loss berubah?



Gradient adalah generalisasi derivative untuk banyak parameter sekaligus.

$$dL/dw \approx \Delta L/\Delta w$$

- ➔ **$dL/dw > 0$**
Naikkan $w \rightarrow$ loss naik. Untuk menurunkan loss, bergerak ke w yang lebih kecil.
- ➔ **$dL/dw < 0$**
Naikkan $w \rightarrow$ loss turun. Untuk menurunkan loss, bergerak ke w yang lebih besar.
- = **$dL/dw \approx 0$**
Dekat titik stasioner; belum tentu minimum global.

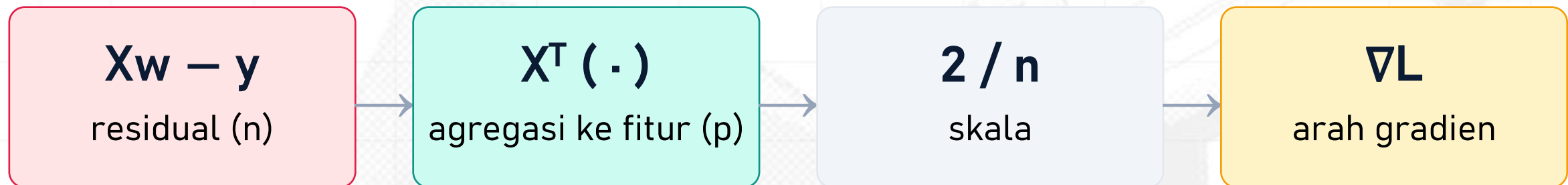
Gradient untuk MSE regresi linear

Derivasi ini adalah jembatan langsung dari least squares ke training iteratif.

$$L(w) = (1/n) \|Xw - y\|^2$$

$$\nabla L(w) = (2/n) X^T (Xw - y)$$

BACA DARI KANAN KE KIRI



Insight

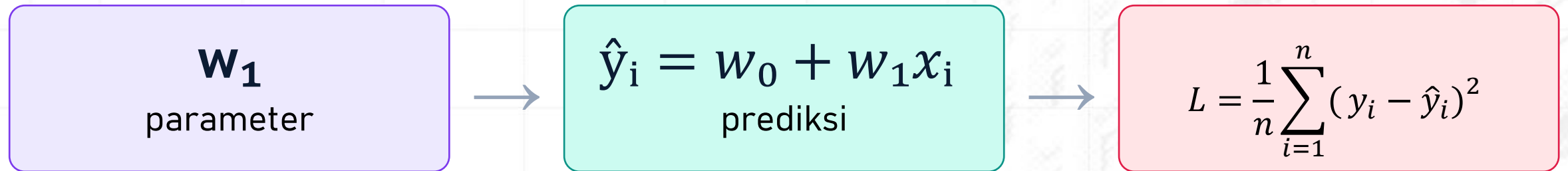
Gradien menggabungkan residual dengan struktur fitur. Fitur yang berkorelasi kuat dengan residual menghasilkan komponen gradien besar.

Cek dimensi

X: $n \times p$, residual: $n \times 1$, $X^T r$: $p \times 1$. Gradien selalu berdimensi sama dengan parameter w .

Derivasi satu parameter: slope w_1

Untuk $\hat{y}_i = w_0 + w_1 x_i$ gunakan *chain rule*: loss $\rightarrow$ prediksi $\rightarrow$ parameter.



Turunan terhadap prediksi $\frac{\partial \hat{y}_i}{\partial w_1} = x_i$

Turunan terhadap loss prediksi $\frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{n} (y_i - \hat{y}_i)$

Faktor pertama

Seberapa sensitif loss terhadap prediksi. Membesar bila residual besar.

Faktor kedua

Seberapa sensitif prediksi terhadap parameter. Untuk bias w_0 nilainya 1, sehingga $\partial L / \partial w_0 = (-2/n) \sum (y_i - \hat{y}_i)$.

$$\frac{\partial L}{\partial w_1} = \sum_{i=1}^n \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_1}$$

$$\frac{\partial L}{\partial w_1} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i)$$

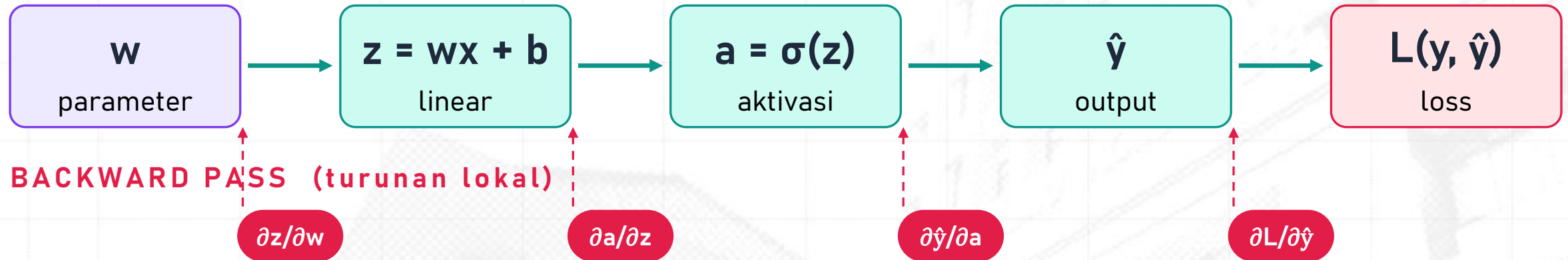
$$w_1 \rightarrow \hat{y}_i \rightarrow L \rightarrow \frac{\partial L}{\partial w_1} \rightarrow w_1^{new}$$

$$w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1} \text{ dengan } \eta = \text{learning rate.}$$

Chain rule adalah inti *backpropagation*

Neural network hanya memperpanjang rantai dependensi.

FORWARD PASS



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}$$

$$\frac{\partial L}{\partial \hat{y}}$$

loss → output

$$\frac{\partial \hat{y}}{\partial a}$$

output → aktivasi

$$\frac{\partial a}{\partial z}$$

turunan aktivasi

$$\frac{\partial z}{\partial w}$$

linear → bobot

**Gradient descent + chain rule =
fondasi deep learning**

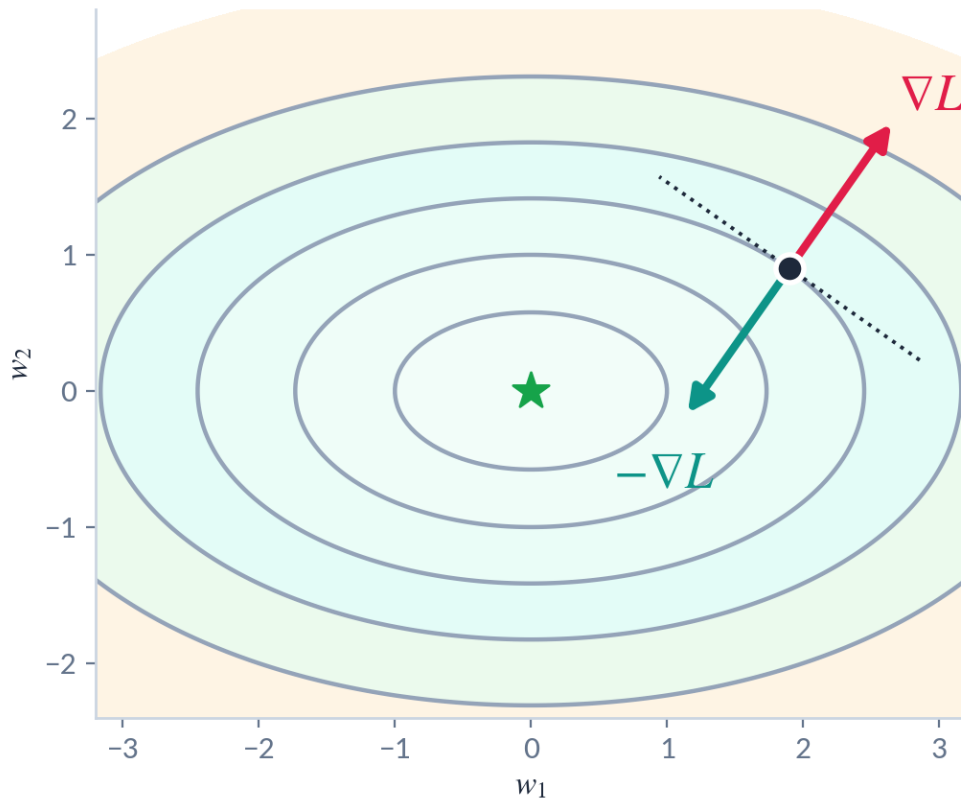
Rantai bisa berisi ratusan lapisan;
prinsip perhitungannya sama.

Contoh: jika turunan lokal = 2, 0.5, 0.25, dan 3 → $\frac{\partial L}{\partial w} = 2 \times 0.5 \times 0.25 \times 3 = 0.75$

Backpropagation = chain rule yang dihitung dari kanan ke kiri untuk memperoleh gradien setiap parameter.

Gradien sebagai arah di ruang parameter

Gradien menunjuk kenaikan tercepat; kita bergerak ke arah sebaliknya.



∇L : arah kenaikan tercepat



Menunjuk ke arah loss bertambah paling cepat secara lokal.

$-\nabla L$: arah penurunan tercepat



Dipakai gradient descent untuk memperbarui parameter.

$\nabla L \perp$ kontur



Gradien tegak lurus terhadap garis dengan nilai loss yang sama.

Gradient check



Verifikasi implementasi: $(L(w+\epsilon) - L(w-\epsilon)) / 2\epsilon$ harus $\approx \partial L / \partial w$ untuk ϵ kecil (mis. $1e-5$).

04

OPTIMIZATION

Gradient Descent: Iterative Learning

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w^{(t)})$$

- 1 Anatomi satu update
- 2 Learning rate & stabilitas
- 3 Feature scaling & conditioning
- 4 Batch, SGD, mini-batch
- 5 Konveks vs non-konveks
- 6 Momentum dan Adam
- 7 Implementasi NumPy

Satu update *gradient descent*

Tiga komponen menentukan langkah berikutnya.

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w^{(t)})$$



Current state: $w^{(t)}$

Posisi saat ini di ruang parameter.



Direction: $-\nabla L$

Arah penurunan lokal tercepat.



Step size: η

Seberapa jauh bergerak pada update tersebut.



Analogi: turun gunung berkabut

Kita tidak melihat seluruh gunung, hanya kemiringan tepat di bawah kaki.

Optimization = keputusan lokal yang diulang-ulang dan diharapkan membawa parameter ke region dengan loss rendah.

Contoh numerik satu langkah

Gunakan model satu parameter $\hat{y} = wx$ tanpa bias agar perhitungan terlihat.

DATA

x : 1 2 3
 y : 2 4 6

Loss: $L(w) = \frac{1}{3} \sum_i (wx_i - y_i)^2$

Turunan: $dL/dw = \frac{2}{3} \sum_i x_i (wx_i - y_i)$

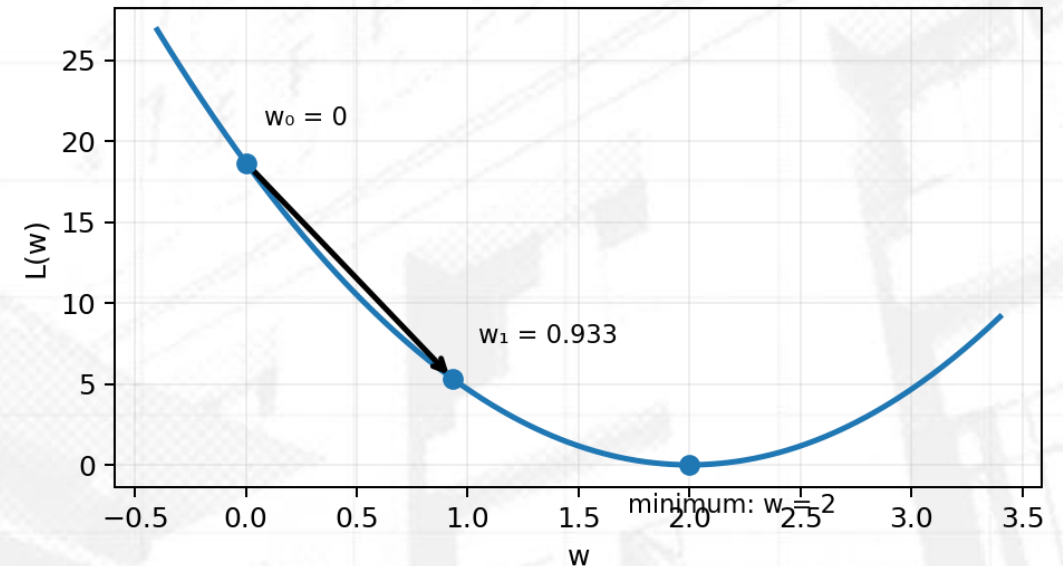
MULAI DARI $w_0 = 0$

1. Hitung gradien pada w_0

$$\begin{aligned} dL/dw &= \frac{2}{3} [1(-2) + 2(-4) + 3(-6)] \\ &= \frac{2}{3} (-28) = -18.67 \end{aligned}$$

2. Update parameter ($\eta = 0.05$)

$$\begin{aligned} w_1 &= w_0 - \eta(dL/dw) \\ &= 0 - 0.05(-18.67) = 0.933 \end{aligned}$$



Loss turun: 18.67 $\rightarrow$ 5.31

Satu langkah membawa w : 0 $\rightarrow$ 0.933, mendekati optimum $w = 2$.

Makna tanda negatif: gradien $< 0 \rightarrow$ update membuat w bertambah.

Beberapa iterasi dan batas stabilitas η

Mengapa η kecil konvergen, sedangkan η terlalu besar dapat berosilasi dan menjauh?

ASAL RUMUS — jangan langsung melompat ke λ

1 Gradient dari contoh sebelumnya

$$dL/dw = \frac{2}{3} \sum x_i (wx_i - y_i)$$

$$\text{karena } y_i = 2x_i \rightarrow dL/dw = \frac{2}{3} \sum x_i^2 (w - 2)$$

$$\sum x_i^2 = 1^2 + 2^2 + 3^2 = 14$$

$$\therefore dL/dw = (28/3)(w - 2)$$

2 Masukkan ke update gradient descent

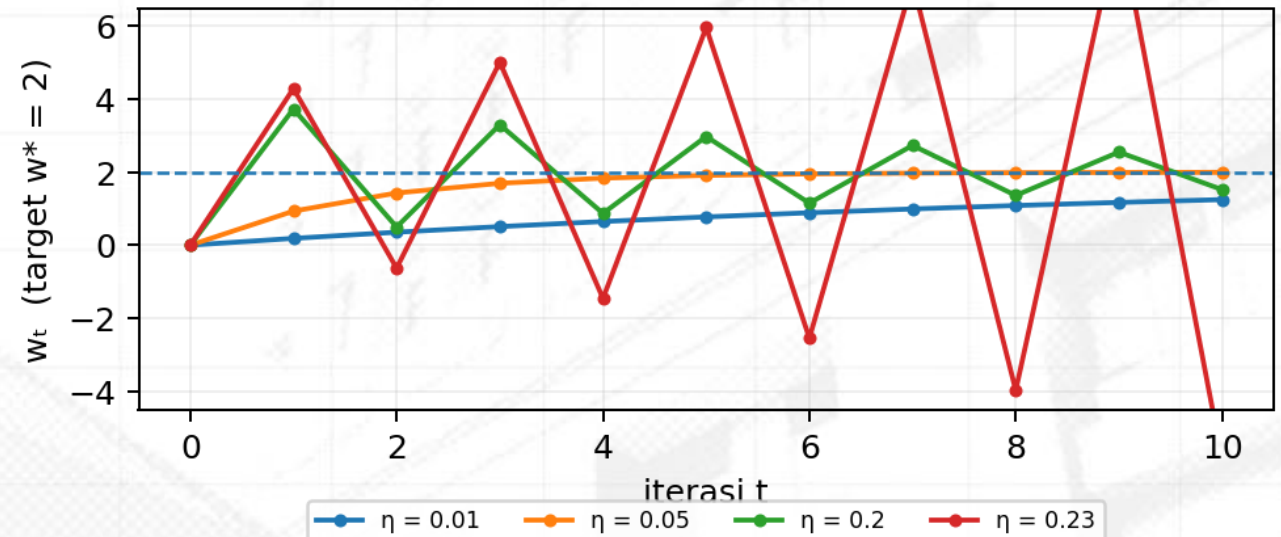
$$w_{t+1} = w_t - \eta (28/3)(w_t - 2)$$

$$w_{t+1} - 2 = [1 - \eta(28/3)](w_t - 2)$$

$$\eta = 0.05 \rightarrow \text{faktor} = 1 - 0.05(28/3) = 0.533$$

Jarak ke optimum tinggal 53.3% tiap iterasi.

Lintasan w untuk beberapa learning rate



BATAS STABIL

$$|1 - \eta(28/3)| < 1 \Rightarrow 0 < \eta < 3/14 \approx 0.214$$

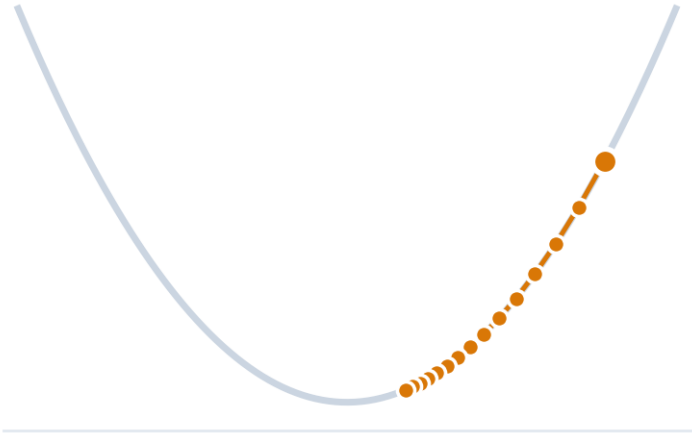
$\eta = 0.23$ melewati batas $\rightarrow$ osilasi membesar.

Intuisi: η menentukan seberapa besar jarak ke optimum dikalikan setiap iterasi.

$|\text{faktor}| < 1 \rightarrow$ jarak mengecil; $|\text{faktor}| > 1 \rightarrow$ menjauh.

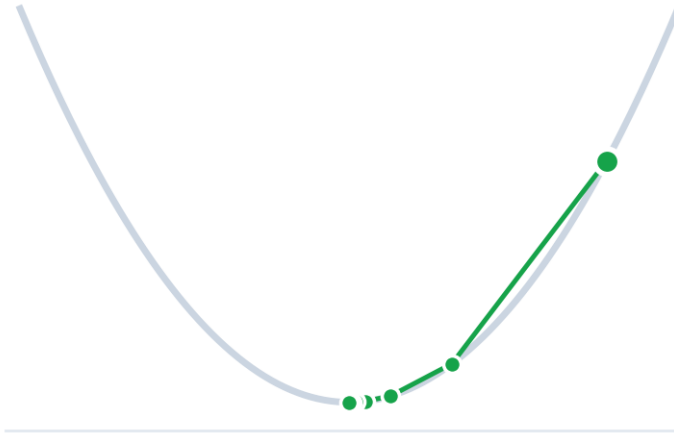
Learning rate: terlalu kecil, tepat, terlalu besar

η mengontrol stabilitas dan kecepatan konvergensi.



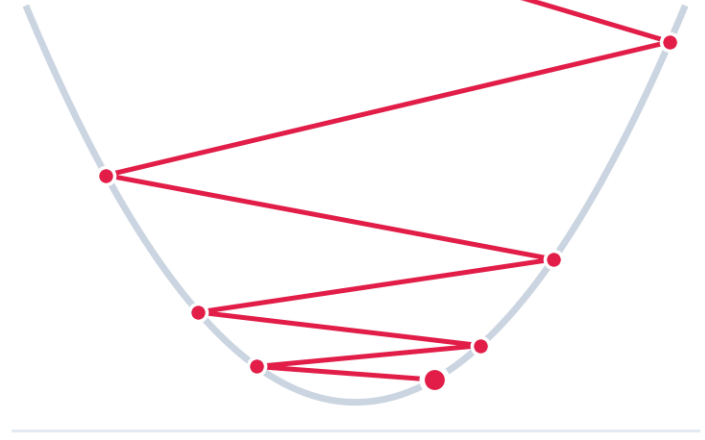
Terlalu kecil

- Stabil, tetapi sangat lambat
- Banyak iterasi, mahal secara komputasi



η sesuai

- Loss turun konsisten
- Konvergensi efisien tanpa overshoot



Terlalu besar

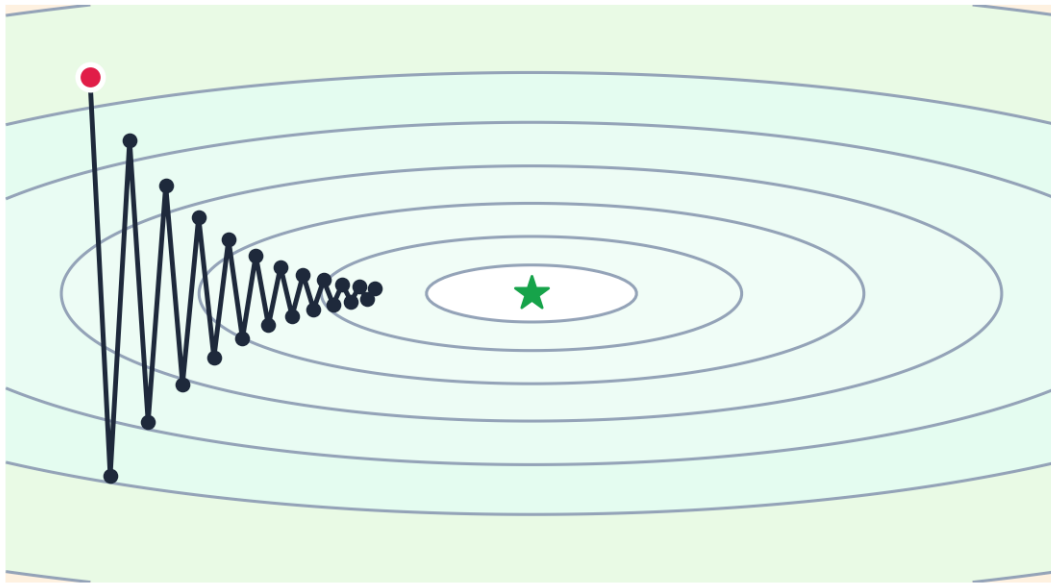
- Melompati minimum, osilasi
- Loss dapat divergen / NaN

$\text{step} = \eta \times \text{gradient}$ • praktik: lr sweep skala log ($1e-4 \dots 1e0$), schedule, warmup

Feature scaling memengaruhi optimasi

Skala fitur mengubah geometri loss landscape.

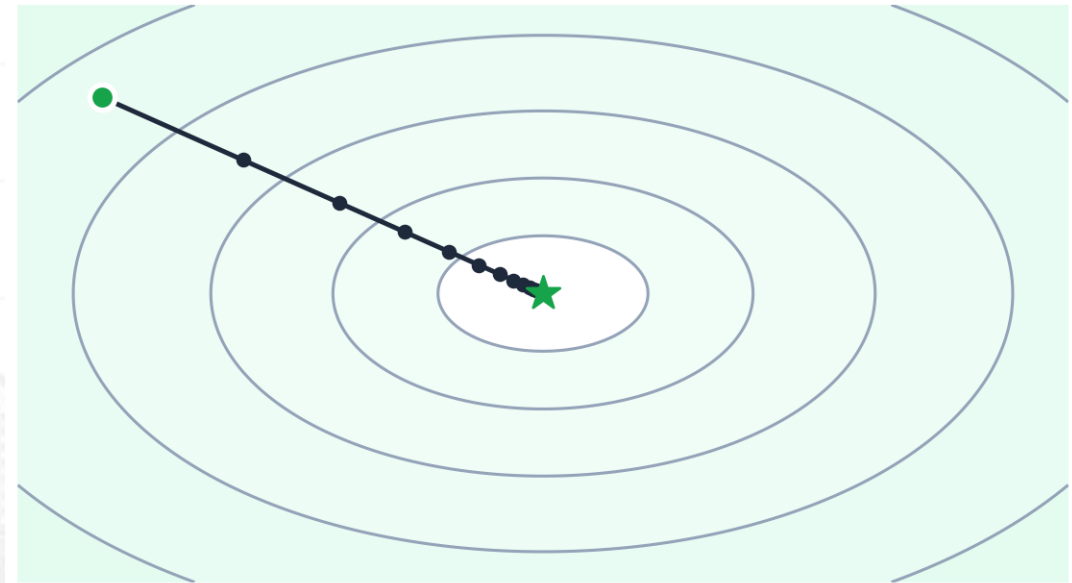
Tanpa scaling: kontur memanjang



$x_1: 0-1, x_2: 0-100000 \rightarrow$ zig-zag dan sensitif terhadap η .

$$x_j' = (x_j - \mu_j) / \sigma_j$$

Setelah standardization: kontur membulat



Skala sebanding $\rightarrow$ landscape isotropik $\rightarrow$ update lebih stabil.

Hitung μ dan σ hanya dari data train, lalu terapkan ke val/test (hindari leakage).

Batch, stochastic, dan mini-batch gradient descent

Yang berubah adalah berapa banyak sampel yang dipakai untuk mengestimasi gradien setiap update.

Batch GD Gradien dari seluruh n data.

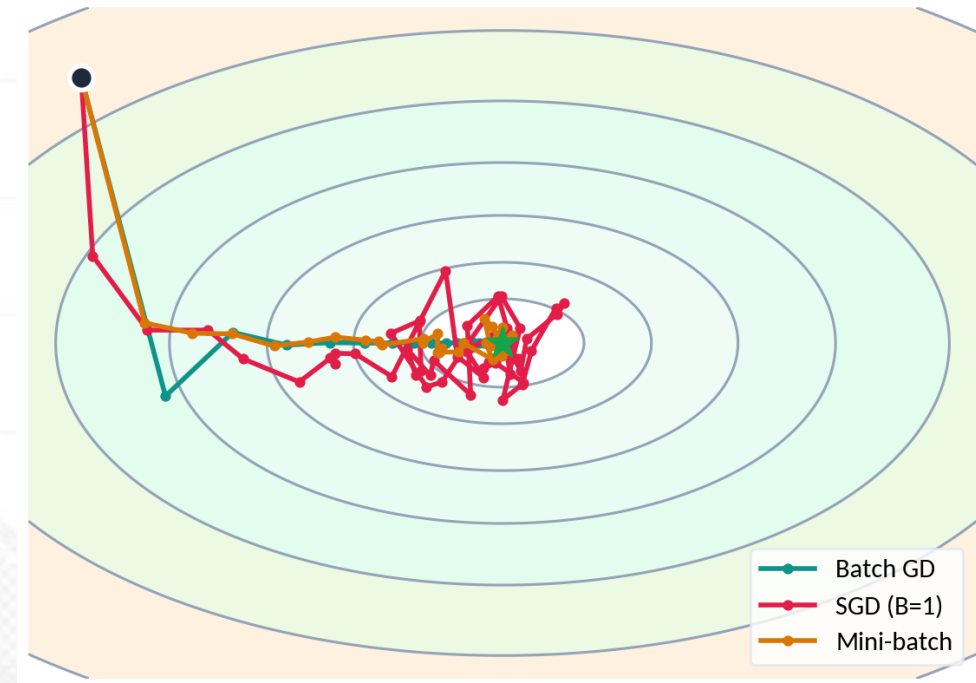
- + stabil
- – mahal untuk data besar; satu update per epoch

Stochastic GD Satu sampel per update.

- + update sangat sering, dapat lolos dari region datar
- – noisy

Mini-batch GD B sampel per update.

- + efisien GPU, noise moderat
- + default deep learning



Mini-Batch Gradient Descent.
$$g = \frac{1}{B} \sum_{i \in \text{batch}} \nabla \ell_i(w)$$

Artinya:

- B = jumlah data dalam satu batch (batch size)
- $\ell_i(w)$ = loss untuk data ke- i
- $\nabla \ell_i(w)$ = gradien loss data ke- i terhadap parameter w
- g = rata-rata gradien seluruh data dalam batch

Epoch, iteration, batch size

Istilah implementasi yang sering membingungkan ketika membaca paper.

DATASET N = 10.000 SAMPEL, DIBAGI MENJADI MINI-BATCH B = 100

batch 1

100 sampel

batch 2

100 sampel

batch 3

100 sampel

...

batch 100

100 sampel

1 epoch = 100 iterations = seluruh sampel terlihat satu kali



Epoch

Satu putaran penuh:
seluruh sampel training
sudah dipakai satu kali.



Iteration

Satu update parameter
memakai satu mini-batch.



Batch size

Jumlah sampel per iterasi
(B). Memengaruhi noise
gradien dan memori.

$\text{iterations per epoch} = [N/B] \rightarrow 10 \text{ epoch} \times 100 = 1.000 \text{ update}$

Kapan *training* berhenti?

“Loss masih turun” tidak otomatis berarti model makin baik untuk data baru.



Fixed epochs

Berhenti setelah jumlah epoch tertentu. Sederhana, tetapi tidak adaptif.

Gradient / loss tolerance

Berhenti ketika perubahan sangat kecil. Cocok untuk optimasi konveks.

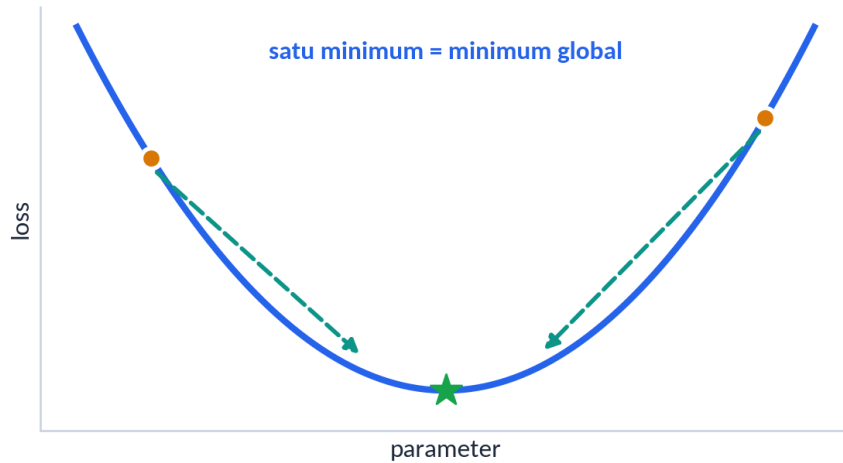
Early stopping

Pantau validation loss; berhenti (dengan patience) saat generalization mulai memburuk.

Stopping criterion adalah bagian dari desain eksperimen, bukan detail implementasi.

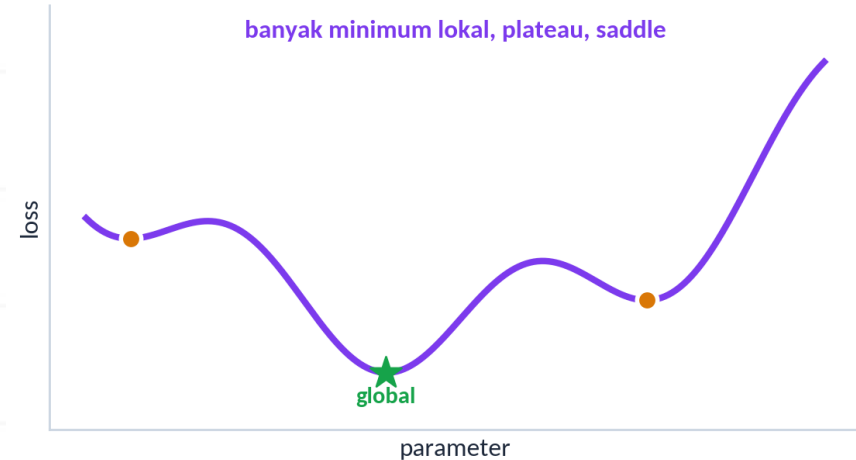
Convex vs non-convex optimization

Regresi linear dengan MSE memiliki landscape konveks; neural network umumnya tidak.



Convex: linear regression + MSE

- Setiap local minimum adalah global minimum
- GD relatif mudah dianalisis
- Masalah utama: conditioning dan step size



Non-convex: deep networks

- Banyak saddle point, flat region, dan simetri parameter
- Tidak ada jaminan sederhana ke global optimum
- Namun solusi loss rendah sering ditemukan secara praktis

Pelajaran: algoritma optimasi harus dibaca bersama struktur objective-nya.

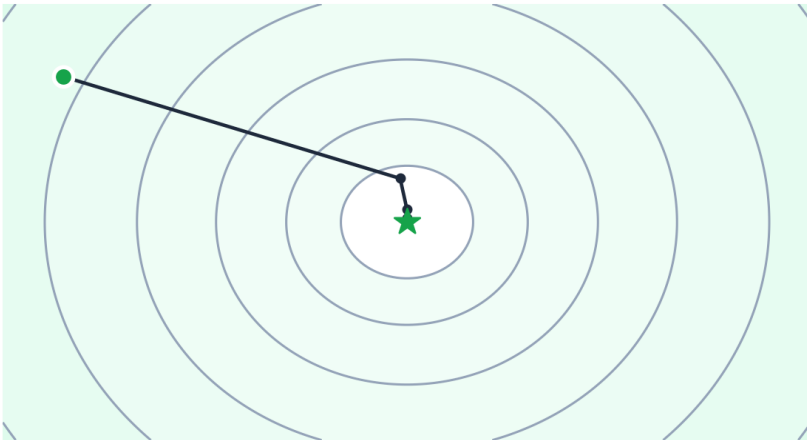
Conditioning: mengapa beberapa problem sulit dioptimasi?

Curvature berbeda antar arah membuat langkah aman di satu arah terlalu kecil atau terlalu besar di arah lain.

$$\kappa(X^T X) = \lambda_{\max} / \lambda_{\min}$$

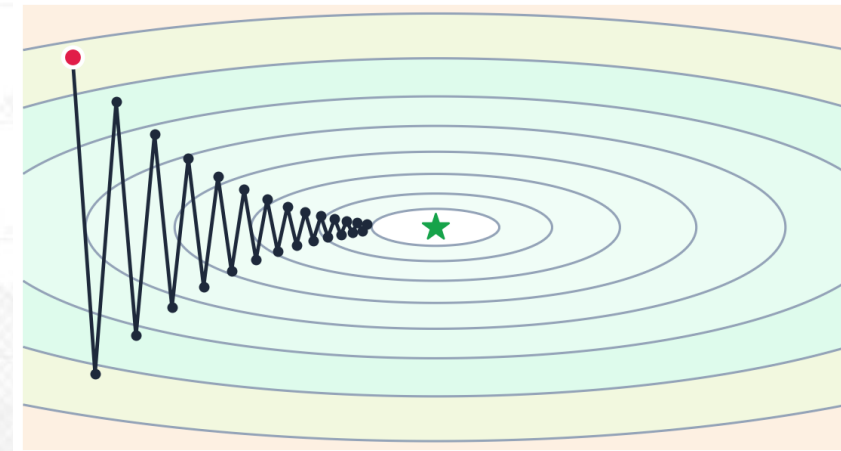
$$\text{laju GD} \approx ((\kappa - 1) / (\kappa + 1))^t$$

Well-conditioned: $\kappa \approx 1$



κ dekat 1 → curvature seragam → solusi numerik stabil dan GD cepat.

Ill-conditioned: $\kappa \gg 1$



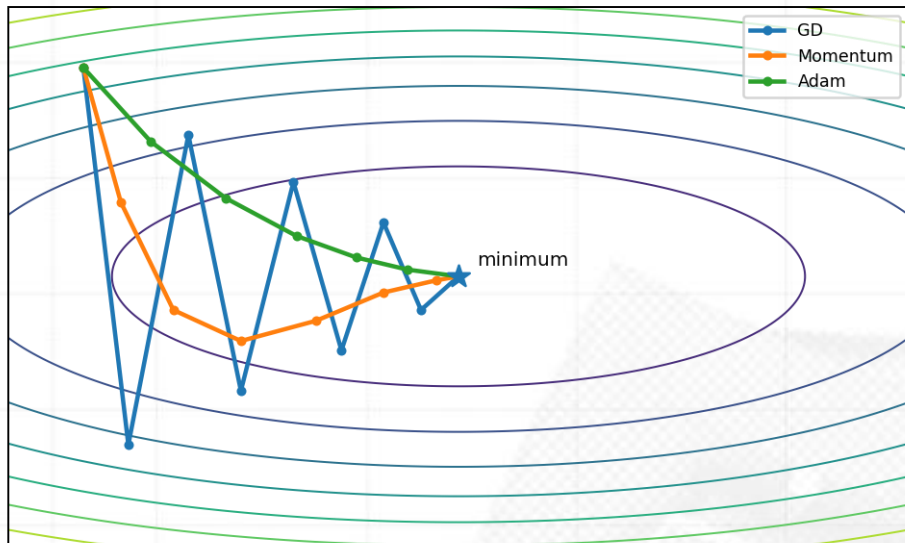
κ besar → fitur redundan / skala berbeda → parameter sensitif dan GD zig-zag lambat.

Scaling, regularization, PCA/SVD, dan optimizer adaptif dapat dilihat sebagai respons terhadap geometri problem.

Dari Gradient Descent ke Momentum dan Adam

Perbedaannya bukan tujuan optimisasi, tetapi bagaimana gradien dipakai untuk menentukan langkah berikutnya.

Intuisi lintasan menuju minimum



GD mudah zig-zag pada lembah sempit.
Momentum mengingat arah sebelumnya;
Adam juga menyesuaikan besar langkah tiap parameter.

1. Gradient Descent

Pakai gradien saat ini saja

$$w \leftarrow w - \eta g$$

η mengatur besar langkah.

2. Momentum

Tambahkan “ingatan” arah gradien sebelumnya

$$v \leftarrow \beta v + g \quad ; \quad w \leftarrow w - \eta v$$

v merangkum arah historis $\rightarrow$ zig-zag berkurang.

3. Adam

Momentum + skala langkah adaptif per parameter

$$m \leftarrow \beta_1 m + (1 - \beta_1) g$$

$$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$$

$$w \leftarrow w - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

m = rata-rata gradien; v = rata-rata gradien² $\rightarrow$ langkah otomatis diskalakan.

Urutan ide: GD = gradien sekarang $\rightarrow$ Momentum = gradien + memori arah $\rightarrow$ Adam = memori arah + penyesuaian ukuran langkah.

05

MODEL CAPACITY

From Linear to Nonlinear Relationships

$$\hat{y} = w_0 + w_1x + \dots + w_dx^d$$

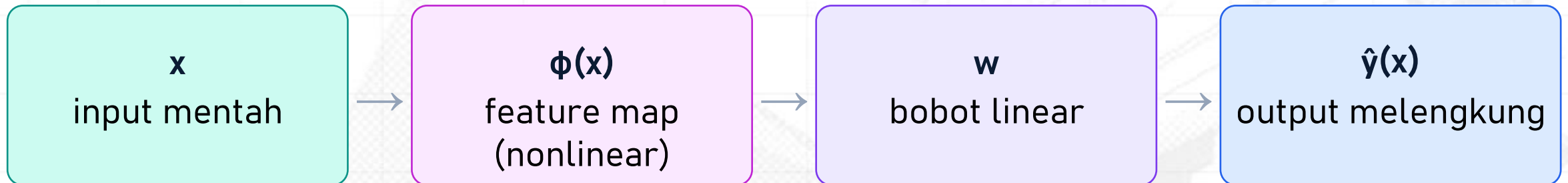
- 1 Polynomial = linear pada fitur $\phi(x)$
- 2 Underfitting pada data melengkung
- 3 Model capacity dan degree
- 4 Training error bisa dibuat ≈ 0

Polynomial regression = linear model pada fitur baru

Nonlinear terhadap x , tetapi tetap linear terhadap parameter w .

$$\phi(x) = [1, x, x^2, x^3]^T$$

$$\hat{y} = w^T \phi(x)$$



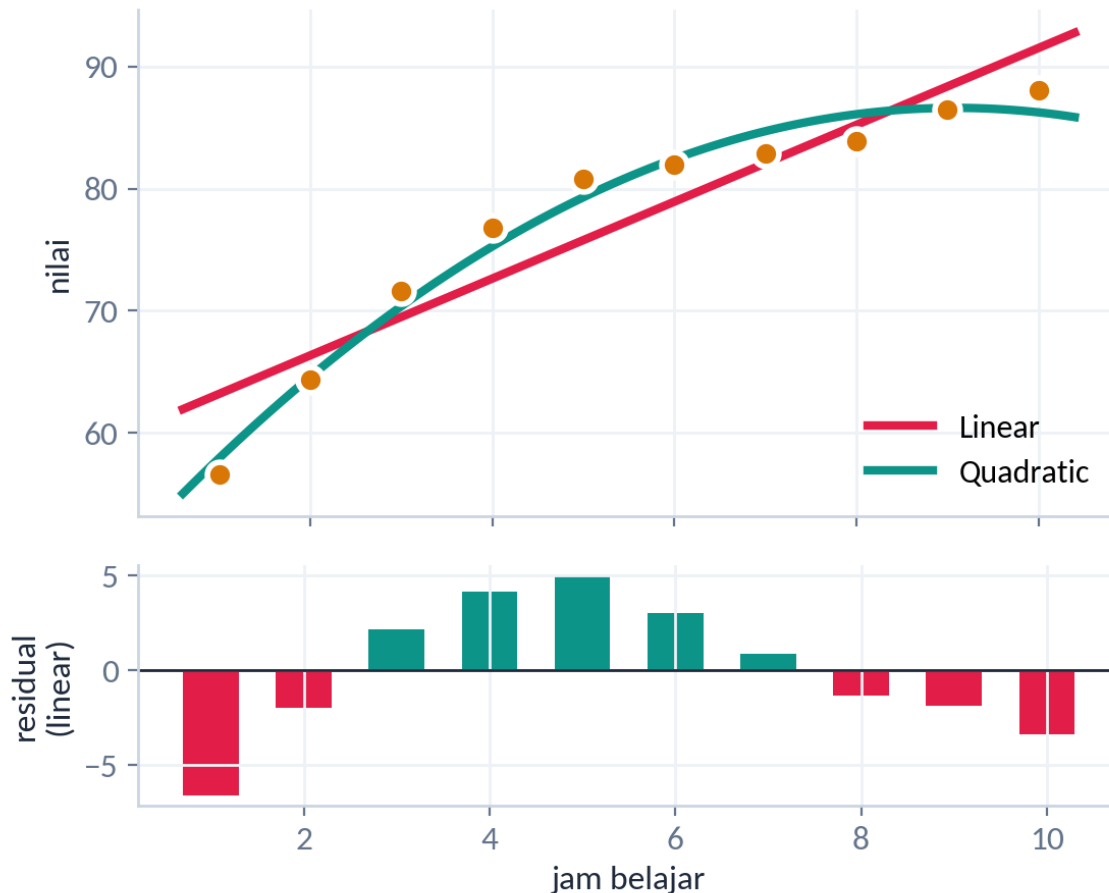
Banyak model “nonlinear” dapat dipahami sebagai representasi fitur nonlinear diikuti kombinasi linear. Neural networks belajar feature map tersebut, bukan menetakannya manual.

Tetap least squares

Matriks desain menjadi $X = [1 \ x \ x^2 \ x^3]$ (Vandermonde). Normal equation dan gradient descent bekerja tanpa perubahan.

Contoh: hubungan belajar dan nilai dapat melandai

Tambahan jam belajar memberi manfaat besar di awal, lalu diminishing returns.



Linear: $\hat{y} = w_0 + w_1x$

Terlalu sederhana. Tidak dapat menangkap curvature → pola residual sistematis → underfitting.

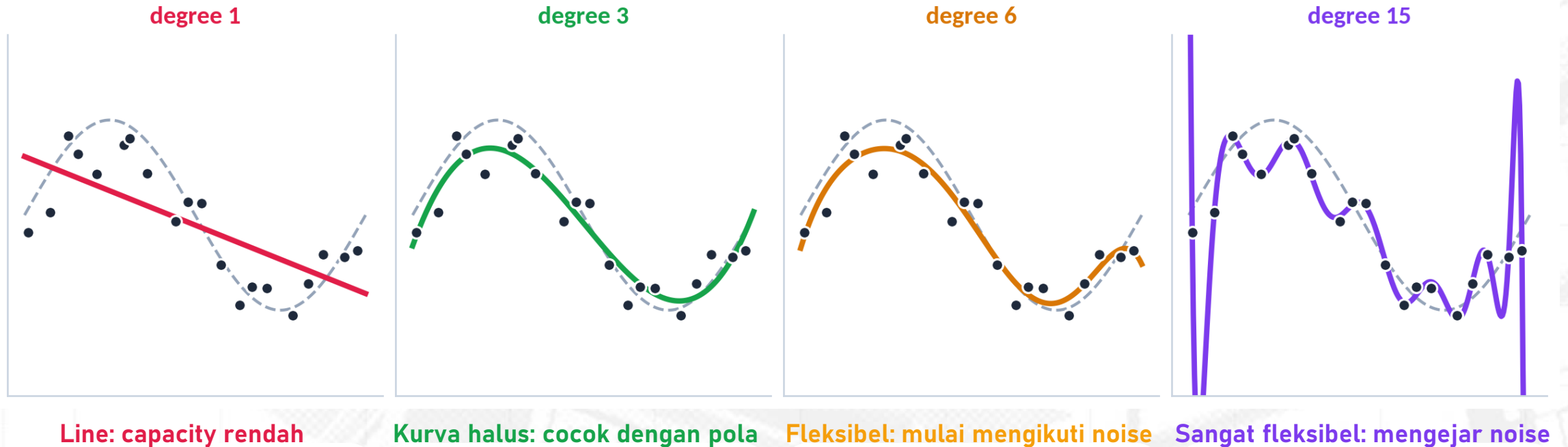
Quadratic: $\hat{y} = w_0 + w_1x + w_2x^2$

Lebih fleksibel dan menangkap curvature. Namun degree terlalu tinggi mulai mengejar noise.

Pertanyaan ilmiahnya bukan “mana yang paling kompleks?”, tetapi “kompleksitas apa yang didukung data?”

Model capacity: kemampuan merepresentasikan pola

Menaikkan degree memperbesar ruang fungsi yang dapat dipilih model.
Garis putus abu = fungsi sebenarnya.



Trade-off

Capacity terlalu rendah → bias tinggi. Terlalu tinggi → variance tinggi. Generalization terbaik sering berada di tengah.

06

GENERALIZATION

Generalization: Fit the Signal, Not the Noise

Expected risk $\neq$ training loss

- 1 Train, validation, test
- 2 Cross-validation
- 3 Underfitting vs overfitting
- 4 Bias-variance
- 5 Double descent
- 6 Data leakage

Train, validation, test: tiga peran berbeda

Pemisahan bukan formalitas; ia menjaga estimasi generalization tetap jujur.

TRAIN 60%

**VALIDATION
20%**

TEST 20%



Train

Belajar parameter w .
Optimizer “melihat” data ini berkali-kali.



Validation

Memilih hyperparameter: λ , degree, learning rate, epoch.
Dipakai untuk keputusan model.



Test

Estimasi final pada data yang tidak dipakai memilih model.
Idealnya dibuka sekali di akhir.

Test dipakai berulang = validation

Jika test dipakai berulang untuk memilih model, ia berubah fungsi menjadi validation set dan estimasi akhir menjadi optimistis.



Aturan praktis

Rasio 60/20/20 atau 80/10/10. Data kecil: gunakan cross-validation.

Cross-validation: memakai data secara efisien

K-fold: setiap sampel menjadi data validasi tepat satu kali.

skor per fold

Fold 1	val	train	train	train	train	S_1
Fold 2	train	val	train	train	train	S_2
Fold 3	train	train	val	train	train	S_3
Fold 4	train	train	train	val	train	S_4
Fold 5	train	train	train	train	val	S_5

Kapan dipakai

Data terbatas atau saat memilih hyperparameter (λ , degree).

Varian split

Stratified untuk kelas tidak seimbang; group untuk subjek yang sama; time-series untuk urutan waktu.

Biaya & jebakan

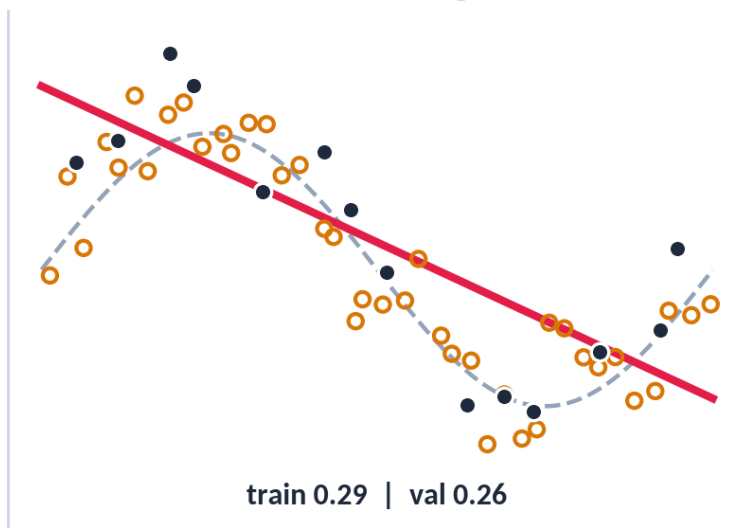
k kali training. Preprocessing harus dilakukan di dalam tiap fold.

$$CV = (1/k) \sum_k S_k$$

Underfitting vs overfitting

Dua kegagalan yang berbeda membutuhkan solusi yang berbeda. Titik hitam = train, lingkaran oranye = validation.

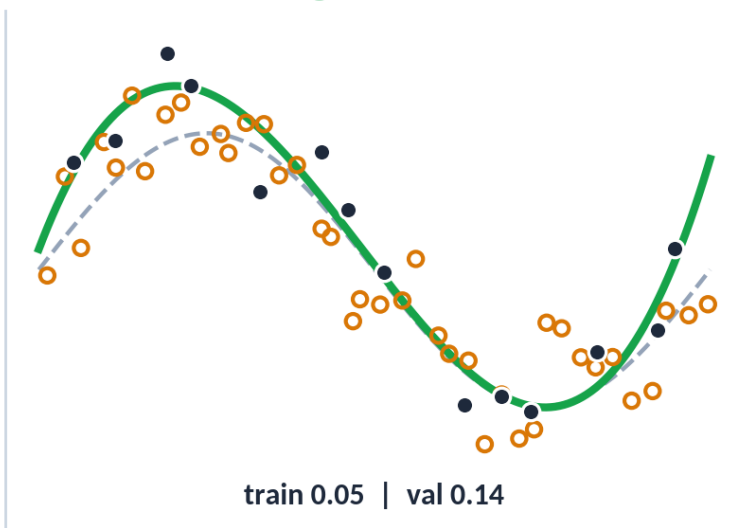
Underfitting



Train error tinggi, validation tinggi

Model terlalu sederhana, fitur kurang informatif, atau optimasi belum cukup.

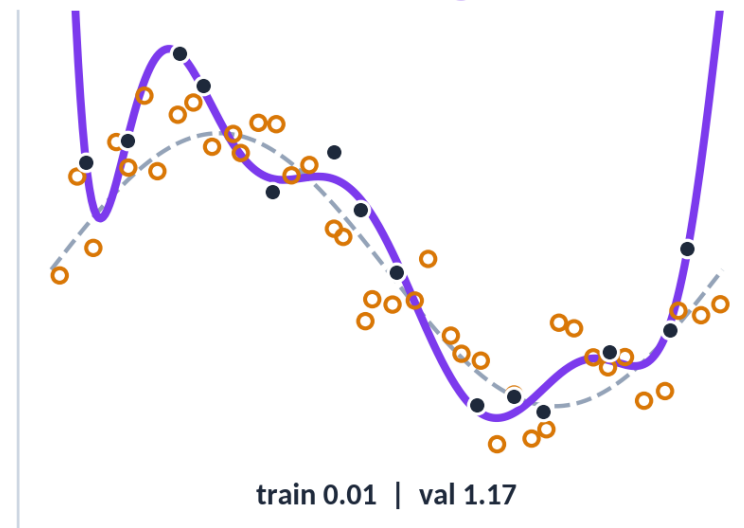
Good generalization



Train rendah, validation juga rendah

Gap wajar dan stabil.

Overfitting



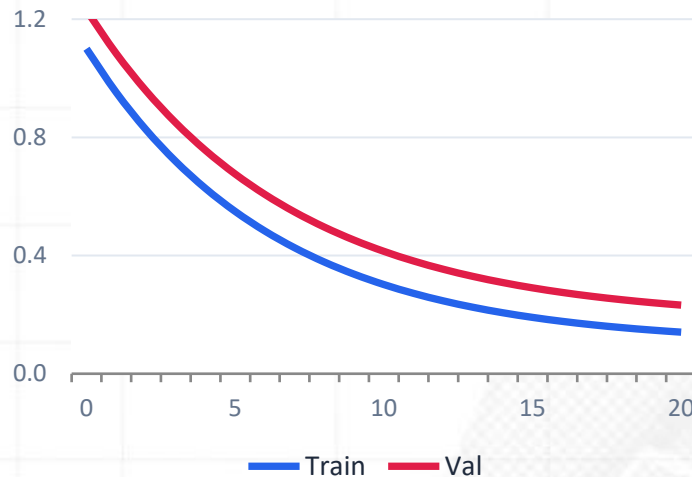
Train sangat rendah, validation lebih tinggi

Model menangkap noise

generalization gap = validation loss – training loss

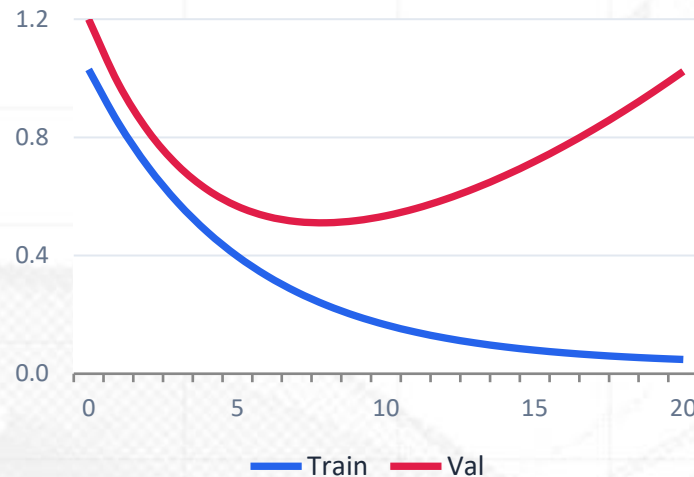
Bagaimana membaca kurva training?

Kurva train-validation adalah alat diagnosis, bukan sekadar gambar wajib di paper. Kurva di bawah bersifat ilustratif.



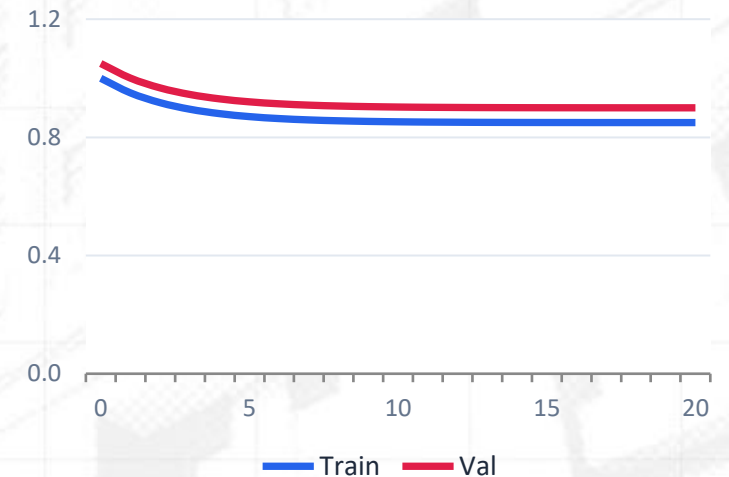
Train ↓, Val ↓

Belajar berjalan baik.
Lanjutkan sampai
improvement validation
melambat.



Train ↓, Val ↑

Overfitting mulai terjadi.
Pertimbangkan early
stopping, regularization,
atau tambah data.



Train tinggi, Val tinggi

Underfitting atau optimasi
gagal. Periksa capacity,
fitur, dan learning rate.

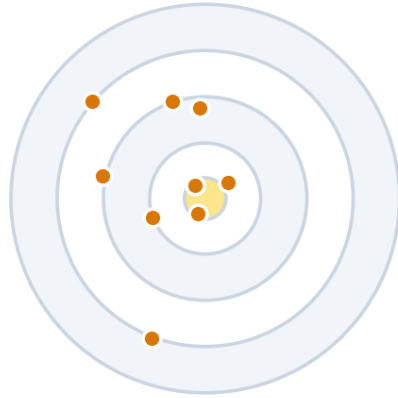
Bias-variance intuition

Error prediksi adalah kompromi antara asumsi terlalu kaku dan sensitivitas berlebihan terhadap sampel.

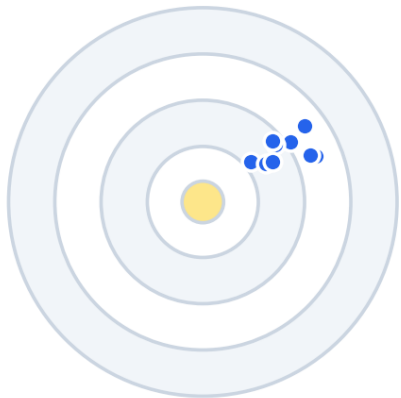
Bias rendah, variance rendah



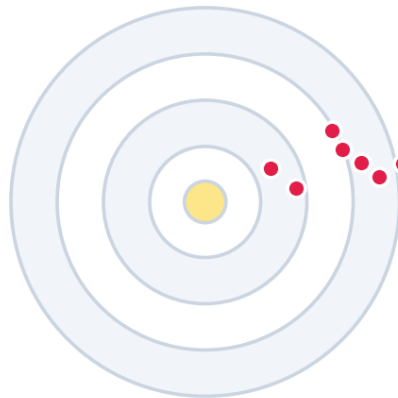
Bias rendah, variance tinggi



Bias tinggi, variance rendah



Bias tinggi, variance tinggi



High bias

Model terlalu kaku. Dataset berubah sedikit → prediksi relatif sama, tetapi secara sistematis meleset. Contoh: garis lurus untuk pola sangat melengkung.

High variance

Model terlalu sensitif. Dataset training berubah sedikit → model berubah besar. Contoh: polynomial degree tinggi pada data sedikit.



Regularization menggeser kompromi dengan sengaja: sedikit bias tambahan untuk menurunkan variance.

Regularization mengubah objective

Kita tidak lagi hanya bertanya “seberapa fit?”, tetapi juga “seberapa kompleks parameter yang diperlukan?”

$$J(w) = L_{\text{data}}(w) + \lambda \Omega(w)$$



Data loss

Kesesuaian dengan observasi.



$\Omega(w)$

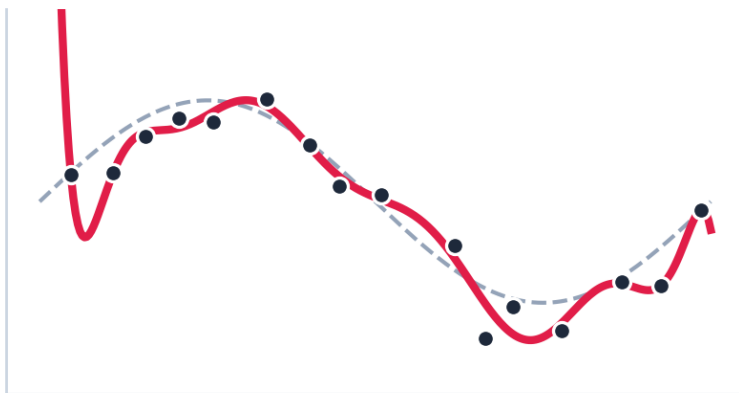
Penalti kompleksitas parameter.



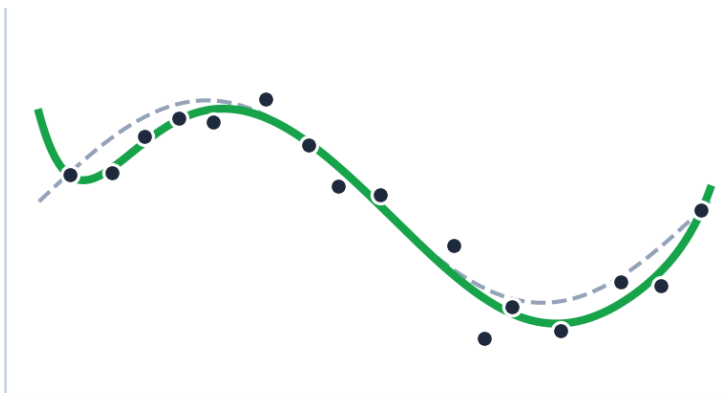
λ

$\lambda = 0$: tanpa regularisasi. λ terlalu besar: underfit.

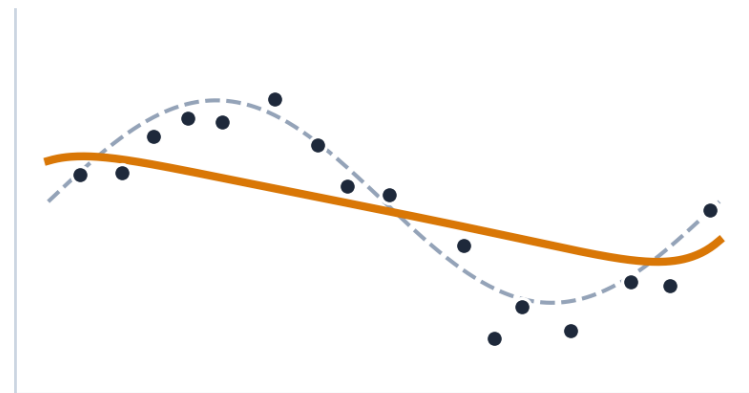
$\lambda = 0$ (overfit)



λ sesuai



λ terlalu besar (underfit)



Terima Kasih

Yuita Arum Sari, S.Kom., M.Kom., Ph.D.
yuita@ub.ac.id

Versi Dokumen:

Tanggal: 15; Bulan: September; Tahun: 2026

© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id